



The Rise of Abstraction in Dag Authoring

From YAML to Minecraft



Volker Janz



MOTIVATION

WHY ABSTRACTION?

Bottleneck?

ASTRONOMER

Everyone *needs* orchestration



Abstraction!

ASTRONOMER



Abstraction!

ASTRONOMER

Abstraction *enables* everyone

Decouple

WHAT A PIPELINE DOES

from

HOW IT IS AUTHORED



Data Engineering

Abstraction

Airflow Dags



attribution_report

crm_newsletter

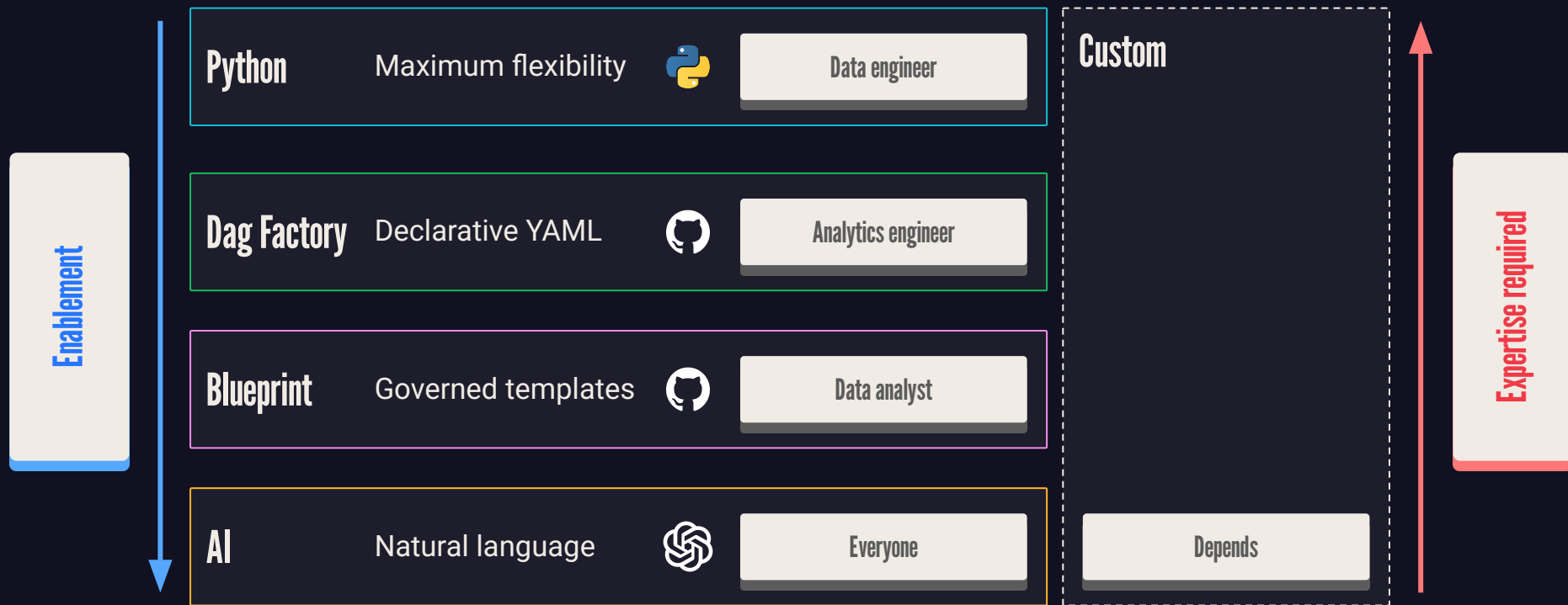
data_import



ABSTRACTION

OVERVIEW

Abstraction ladder



Abstraction ladder



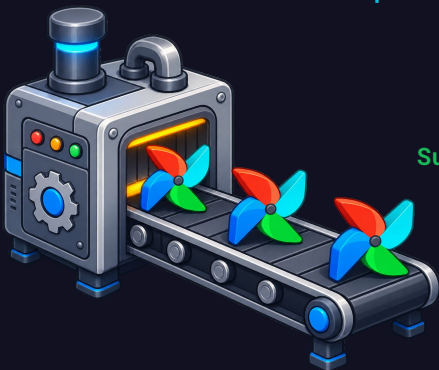
	PYTHON	DAG FACTORY	BLUEPRINT	AI
Required Python knowledge	Very high	Medium	Very Low 🌟	Medium
Required Airflow knowledge	Very high	Medium	Low	Medium
Flexibility	Very high 🌟	High	Medium	High
Governance	Low	Medium	Very high 🌟	Medium
Time-to-value	Slow	Medium	Fast	Fast

DAG FACTORY

DECLARATIVE YAML

DAG FACTORY

DAG Factory is an open source tool
that allows you to dynamically
generate Dags from YAML



Separate logic from structure

Support for TaskFlow API

Custom arguments

Passing data between tasks



revenue_report.yml

```
revenue_report:
  default_args:
    owner: "astronomer"
    start_date: 2026-09-01
    schedule: "@daily"
    template_searchpath: "/usr/local/airflow/include"

  tasks:
    aggregate:
      operator: airflow.providers.common.sql.operators.sql.SQLExecuteQueryOperator
      conn_id: postgres_default
      sql: "sql/aggregate_payments.sql"
      parameters:
        reference_date: "{{ logical_date }}"

    format_report:
      decorator: airflow.sdk.task
      python_callable: include.tasks.revenue_report.format_report
      report_data: "{{ ti.xcom_pull(task_ids='aggregate') }}"
      dependencies: [aggregate]

    send_report:
      decorator: airflow.sdk.task
      python_callable: include.tasks.revenue_report.send_report
      report: +format_report
      dependencies: [format_report]
```

DAG FACTORY

DECLARATIVE Tell **what** result you want

IMPERATIVE Tell **how** to achieve the result

Define outline, no logic (*mostly*)

DAG Factory is **declarative** in syntax, but **imperative** in nature

Still using low-level Airflow components (*operators, decorators*)

ASTRONOMER



revenue_report.yml

```
revenue_report:
  default_args:
    owner: "astronomer"
    start_date: 2026-09-01
  schedule: "@daily"
  template_searchpath: "/usr/local/airflow/include"

tasks:
  aggregate:
    operator: airflow.providers.common.sql.operators.sql.SQLExecuteQueryOperator
    conn_id: postgres_default
    sql: "sql/aggregate_payments.sql"
    parameters:
      reference_date: "{{ logical_date }}"

  format_report:
    decorator: airflow.sdk.task
    python_callable: include.tasks.revenue_report.format_report
    report_data: "{{ ti.xcom_pull(task_ids='aggregate') }}"
    dependencies: [aggregate]

  send_report:
    decorator: airflow.sdk.task
    python_callable: include.tasks.revenue_report.send_report
    report: +format_report
    dependencies: [format_report]
```

DAG FACTORY



dags/dag_generator.py

```
# keep import to ensure the DAG Processor parses the file
from airflow.sdk import dag
from dagfactory import load_yaml_dags
load_yaml_dags(globals_dict=globals())
```

aggregate

SQLExecuteQueryOperator

format_report

@task

send_report

@task

ASTRONOMER



revenue_report.yml

```
revenue_report:
  default_args:
    owner: "astronomer"
    start_date: 2026-09-01
  schedule: "@daily"
  template_searchpath: "/usr/local/airflow/include"

tasks:
  aggregate:
    operator: airflow.providers.common.sql.operators.sql.SQLExecuteQueryOperator
    conn_id: postgres_default
    sql: "sql/aggregate_payments.sql"
    parameters:
      reference_date: "{{ logical_date }}"

  format_report:
    decorator: airflow.sdk.task
    python_callable: include.tasks.revenue_report.format_report
    report_data: "{{ ti.xcom_pull(task_ids='aggregate') }}"
    dependencies: [aggregate]

  send_report:
    decorator: airflow.sdk.task
    python_callable: include.tasks.revenue_report.send_report
    report: +format_report
    dependencies: [format_report]
```

DAG FACTORY



More than just basic Dags



Asset-aware scheduling



Configuration inheritance

via defaults.yml

```
dags/
|-- defaults.yml      owner: platform, retries: 2
|-- marketing/
|   |-- defaults.yml  owner: mkt, "0 1 * * *"
|   |-- campaign.yml  inherits both
|-- finance/
|   |-- defaults.yml  owner: fin
|   |-- ledger.yml    schedule from the root
```



assets.yml

```
# asset-aware scheduling, entirely in YAML
refresh_bookings:
  schedule: "@daily"
  tasks:
    load:
      decorator: airflow.sdk.task
      python_callable: include.bookings.load
    outlets:
      - __type__: airflow.sdk.Asset
        name: "bookings"

revenue_report:
  schedule: # the consumer
  - __type__: airflow.sdk.Asset
    name: "bookings"
```



dtm.yml

```
# dynamic task mapping
process:
  decorator: airflow.sdk.task
  python_callable: include.b.process
  partial: {processing_date: "{{ logical_date }}" }
  expand: {value: +generate}
```

BLUEPRINT

GOVERNED TEMPLATES

BLUEPRINT



include/blueprints/extract.py

```
from blueprint import Blueprint, BaseModel, Field, TaskOrGroup
from airflow.sdk import TaskGroup
from airflow.providers.common.sql.operators.sql \
    import SQLExecuteQueryOperator
```

CONTRACT

```
class ExtractConfig(BaseModel):
    conn_id: str = Field(description="warehouse conn")
    n_bookings: int = Field(default=50, ge=1, le=500)
```

BLUEPRINT

```
class ExtractBookings(Blueprint[ExtractConfig]):
    def render(self, config) -> TaskOrGroup:
        with TaskGroup(self.step_id) as g:
            pull = SQLExecuteQueryOperator(task_id="pull",
                                           conn_id=config.conn_id, sql="pull.sql")
            check = SQLExecuteQueryOperator(task_id="check",
                                           conn_id=config.conn_id, sql="check.sql")
            pull >> check
        return g
```

Data engineer

Creates blueprints

Everyone else

Composes Dags



dags/daily_bookings.yml

```
dag_id: daily_bookings
schedule: "@daily"
steps:
  extract:
    blueprint: extract_bookings
    conn_id: warehouse
    n_bookings: 50
  transform:
    blueprint: revenue_report
    depends_on: [extract]
  notify:
    blueprint: slack_notify
    depends_on: [transform]
    channel: "#data"
```

BLUEPRINT



dags/dag_generator.py

```
from pathlib import Path

from blueprint import build_all_airflow_dags
from blueprint.registry import BlueprintRegistry

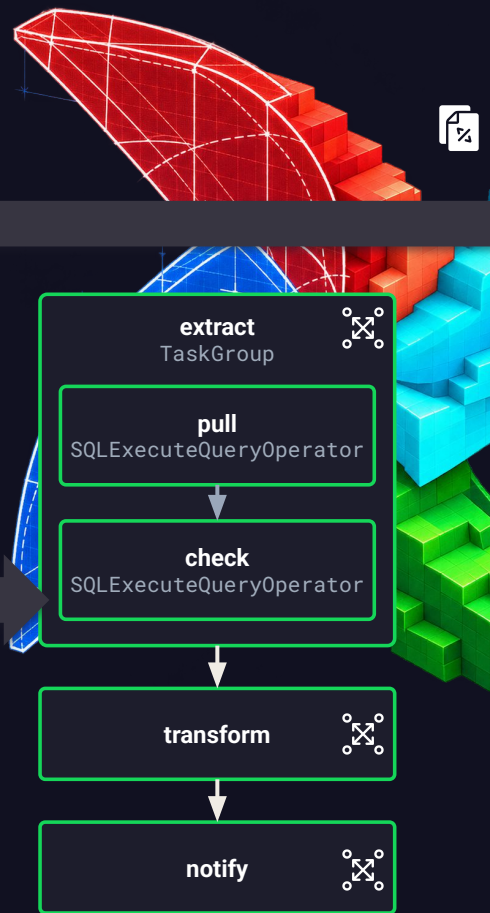
TEMPLATE_DIR = Path("/usr/local/airflow/include/blueprints")
YAML_DIR = Path("/mineflow/blueprints")

_registry = BlueprintRegistry(template_dirs=[TEMPLATE_DIR])
build_all_airflow_dags(
    search_path=YAML_DIR,
    register_globals=globals(),
    bp_registry=_registry
)
```



dags/daily_bookings.yml

```
dag_id: daily_bookings
schedule: "@daily"
steps:
  extract:
    blueprint: extract_bookings
    conn_id: warehouse
    n_bookings: 50
  transform:
    blueprint: revenue_report
    depends_on: [extract]
  notify:
    blueprint: slack_notify
    depends_on: [transform]
    channel: "#data"
```



BLUEPRINT



Versioning

```
# one class per version
class ExtractBookings(Blueprint[ExtractConfig]):
    ... # implicitly version 1

class ExtractBookingsV2(Blueprint[ExtractConfigV2]):
    ... # version 2
```

```
# existing files keep working, opt in per step
steps:
  extract:
    blueprint: extract_bookings
    version: 2
```



Validation

```
steps:
  extract:
    blueprint: extract_bookings
    conn_id: warehouse
    n_bookings: 5000 # limit is 500
    chanel: "#data" # typo
```

```
ValidationError: 2 errors for ExtractConfig

n_bookings
  Input should be ≤ 500

chanel
  Extra inputs are not permitted
```



CLI

```
volker.janz@airflowSummit:~/blueprint

# validate Dag definitions
$ blueprint lint
PASS customer_pipeline.dag.yaml (dag_id=some_dag)

# list available blueprints
$ blueprint list

# describe a blueprint's config schema
$ blueprint describe extract

# describe a specific version
$ blueprint describe extract -v 1

# create new Dag interactively
$ blueprint new
```

MINECRAFT

???

```
make demo

[12:46:27 INFO]: Done preparing level "world" (0.641s)
[12:46:27 INFO]: [Mineflow] Enabling Mineflow v0.1.0
[12:46:28 INFO]: [Mineflow] Mineflow enabled with 'dagfactory' backend! Files saved under: /
Users/volker.janz/projects/mineflow/server/plugins/Mineflow
[12:46:28 INFO]: [spark] Starting background profiler...
[12:46:29 INFO]: Running delayed init tasks
[12:46:29 INFO]: Done (6.870s)! For help, type "help"
[12:46:29 WARN]: *****
*****
[12:46:29 WARN]: You are running the latest build for your Minecraft version (1.21.11)
[12:46:29 WARN]: However, you are 4 release(s) behind the latest stable release (26.2)!
[12:46:29 WARN]: It is recommended that you update as soon as possible
[12:46:29 WARN]: https://papermc.io/downloads/paper
[12:46:29 WARN]: *****
*****
[12:48:32 INFO]: UUID of player vojay is 6a837e90-0d2c-4bcf-9be2-281a78167e64
[12:48:33 INFO]: vojay joined the game
[12:48:33 INFO]: vojay[/127.0.0.1:56543] logged in with entity id 1 at ([world]0.0, -60.0, 0.0)
[12:48:37 INFO]: vojay has made the advancement [Bring Home the Beacon]
[12:48:50 INFO]: [Mineflow] Updated Dag file: 158bae9d-e1f5-4ca0-adeb-93b920667220.yml
> []
```

```
watch -d -n1 tree

Every 1.0s: tree      astroBook.local: Fri Aug 21 12:49:06 2026
                                in 0.043s (0)

.
├── blueprints
├── config.yml
└── dags
    └── 158bae9d-e1f5-4ca0-adeb-93b920667220.yml

3 directories, 2 files
```

Minecraft 1.21.11 - Multiplayer (3rd-party Server)



Airflow

airflow.localhost:6563/dags

MINEFLOW REFRESH: ON EVERY 10 SEC > next in SERVER: 1

Dags Dag Runs Task Instances

Search Dags

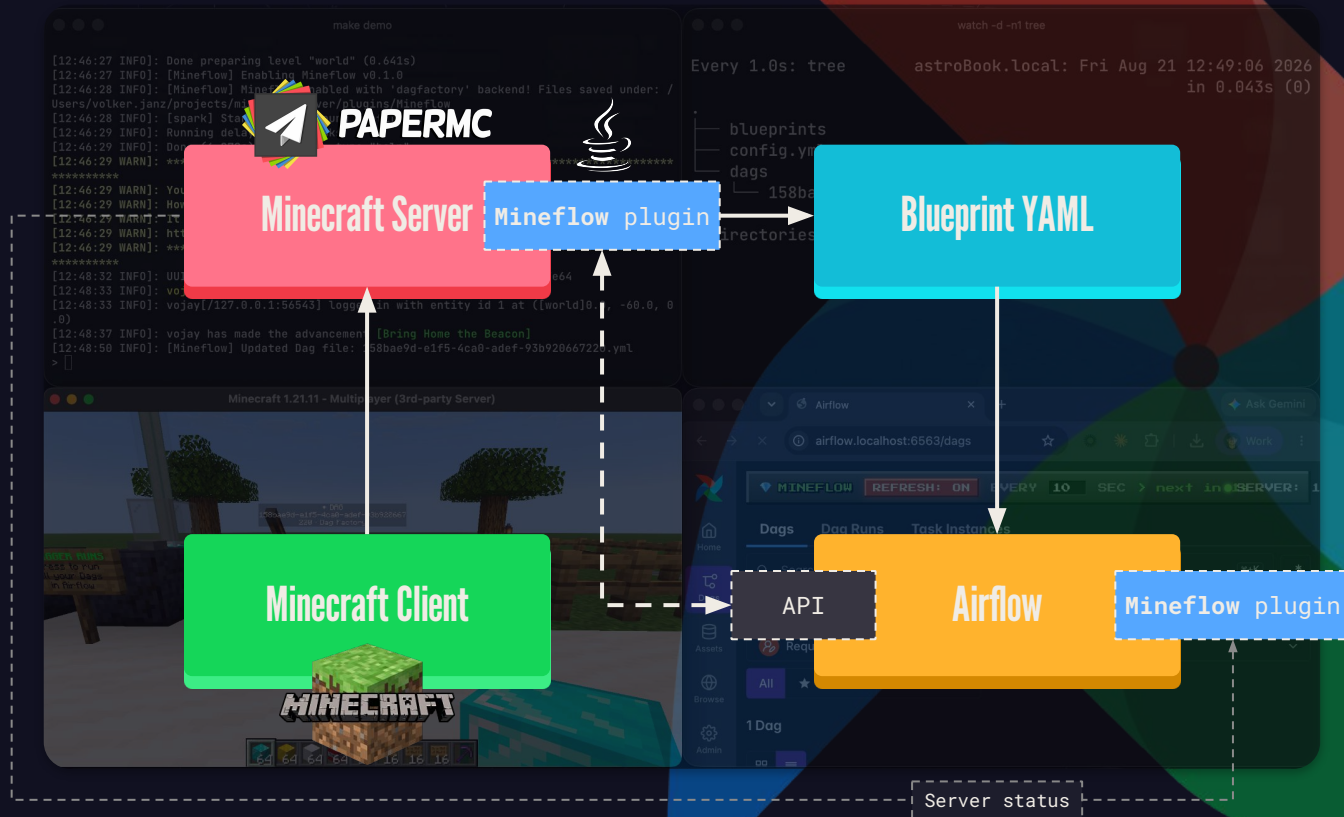
All Failed Queued Running Success

Required Actions All Active Paused Filter by tag

All Favorite Unfavorite

1 Dag

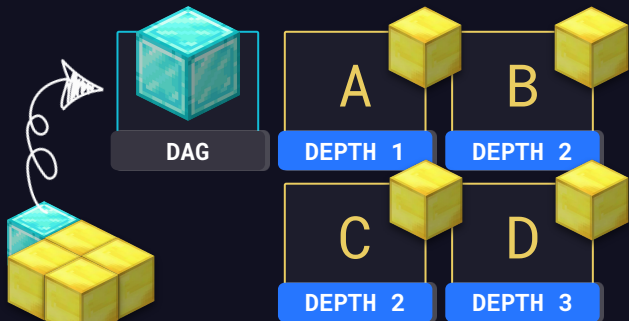
Mineflow



Mineflow



The blocks (top view)



depth = steps from the diamond

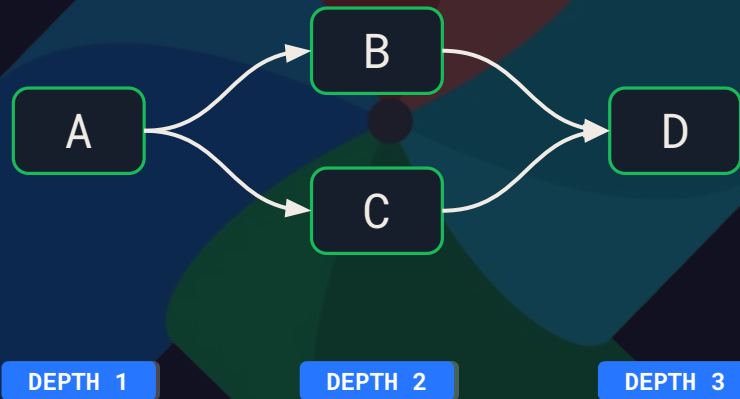
Breadth-first search (BFS)



Traverse through one level of children nodes, then traverse through the level of grandchildren nodes (and so on...)


`recomputeDependencies()`
on every block change

The Dag Airflow sees



Mineflow

GENERAL



DIAMOND

A Dag · sign it to name it

DAG FACTORY



GOLD

Python task · **Dag Factory**

BLUEPRINT



EMERALD

Extract · **Blueprint**



REDSTONE

Load · **Blueprint**



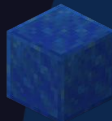
LEVER

Switch Dag Factory ⇌ Blueprint



IRON

Bash task · **Dag Factory**



LAPIS

Transform · **Blueprint**



OBSERVER

Quality · **Blueprint**



LET'S PLAY

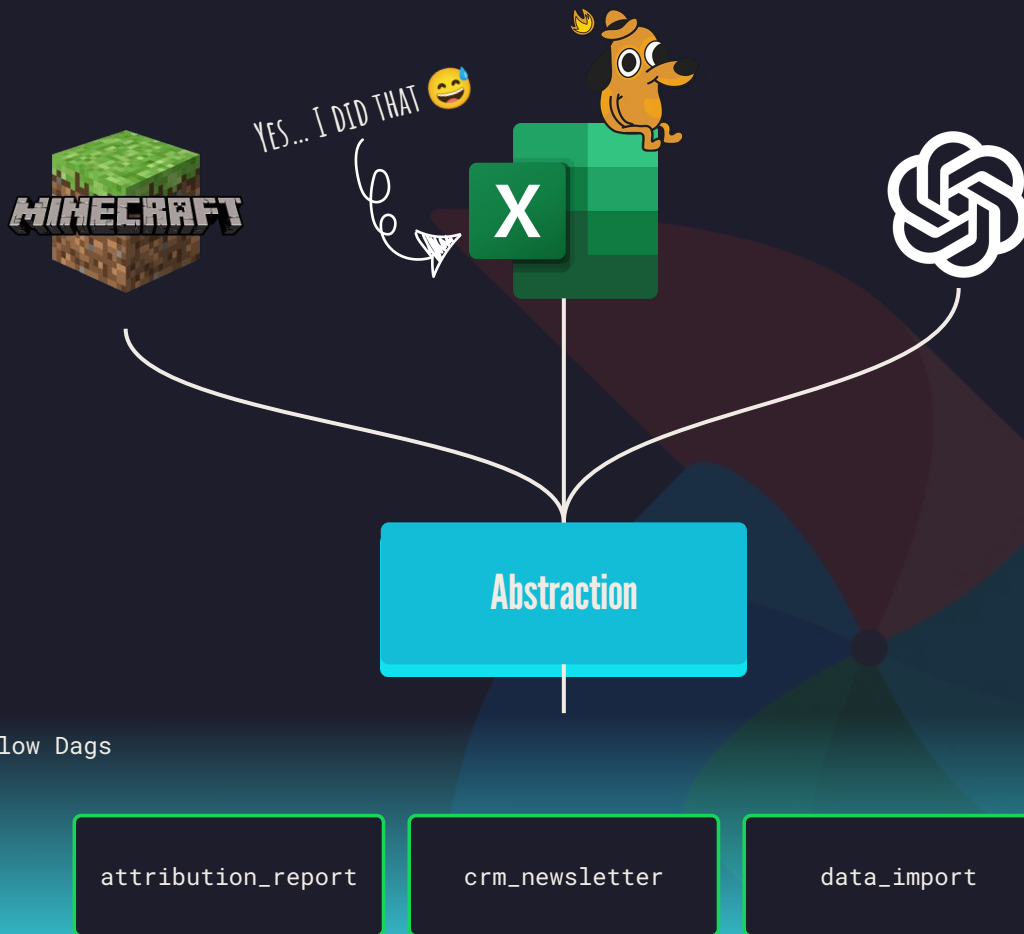
THE MOMENT YOU'VE BEEN WAITING FOR

I WAS WRONG!

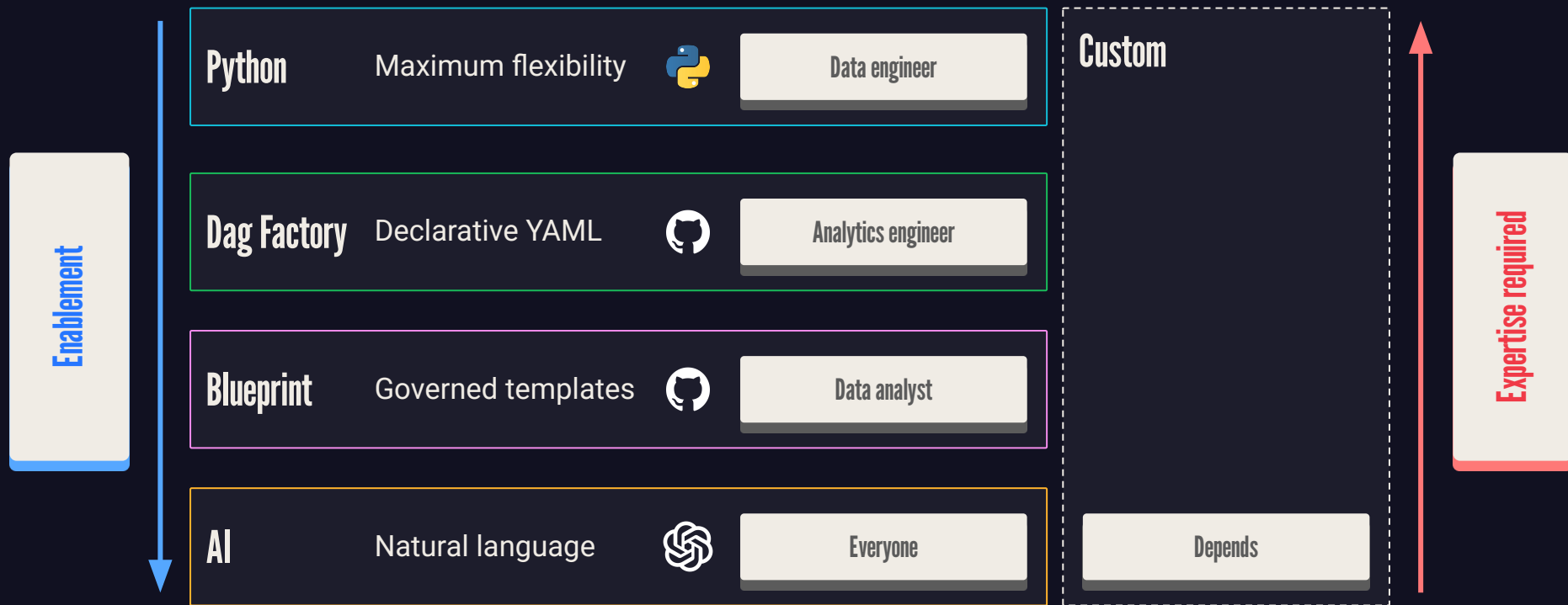
AI IS NOT AN ABSTRACTION LAYER

Everyone and
everything can
be a Dag author

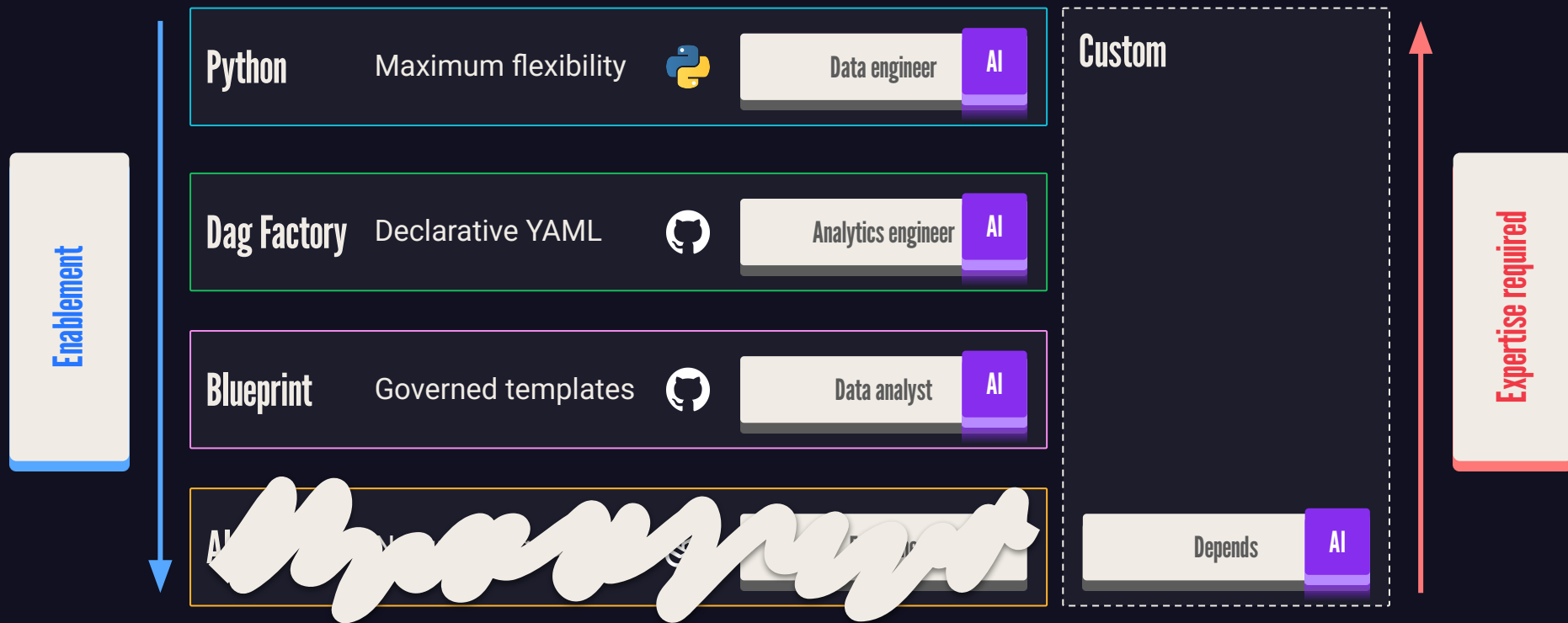
ASTRONOMER



Abstraction ladder

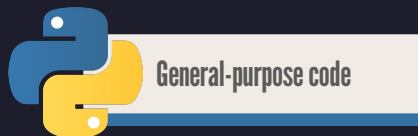
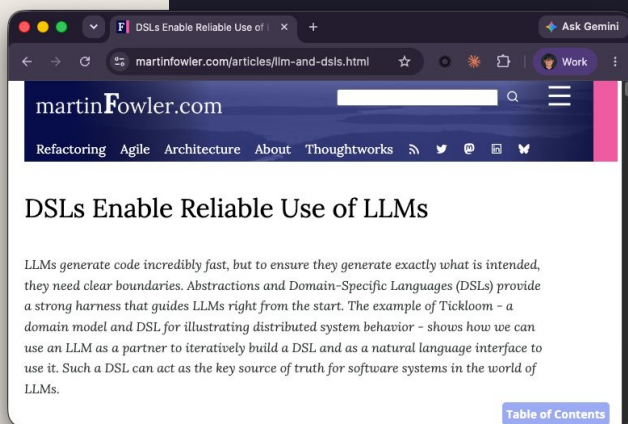


Abstraction ladder

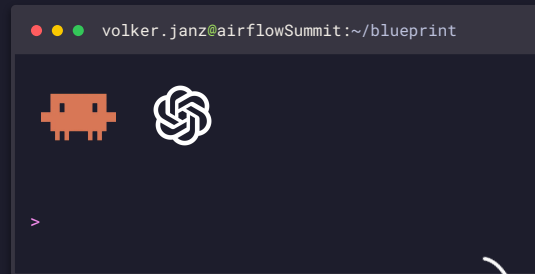


LLMs and DSLs

ASTRONOMER



A huge space of valid programs. Many ways to express one intent, many ways to be subtly wrong.



A narrow grammar a few examples convey whole. Usually ships with a validator: a parser, a JSON Schema, a type checker, a compiler.

GENERATE

LESS LOC TO REVIEW

LESS VARIATION

MORE GUARDRAILS

MORE CONTROL

LESS TOKEN USAGE*

BASED ON: <https://martinfowler.com/articles/llm-and-dsls.html>

LLMs and DSLs

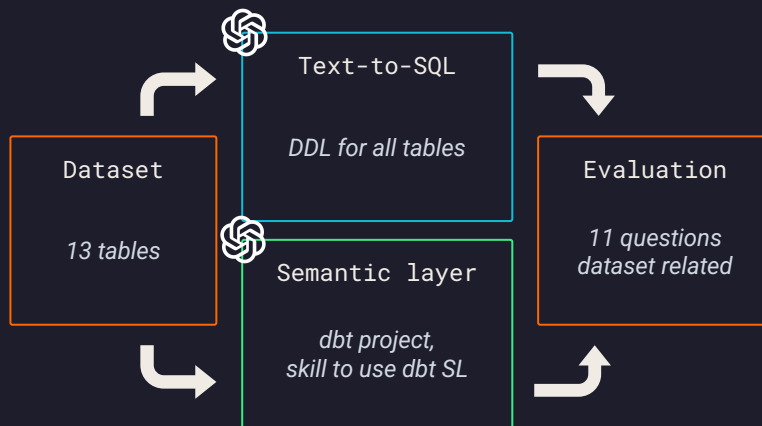
ASTRONOMER

DOMAIN	GENERAL-PURPOSE	DOMAIN-SPECIFIC
Analytics	SQL against raw tables	Semantic layer query
Infrastructure	cloud API calls	Terraform
Orchestration	Python	Blueprint YAML



semantic layer vs. text-to-SQL benchmark

Accuracy results



90.0% → **98.2%**

claude-sonnet-4-6

84.1% → **100%**

gpt-5.3-codex

BASED ON: <https://docs.getdbt.com/blog/semantic-layer-vs-text-to-sql-2026>

AI FOR BLUEPRINT

Blueprint skill



SKILL.md

```
---  
name: blueprint  
description: Define reusable Airflow task group templates with Pydantic  
validation and compose DAGs from YAML. Use when creating blueprint  
templates, composing DAGs from YAML, validating configurations, or  
enabling no-code DAG authoring for non-engineers.  
---
```

Blueprint Implementation

You are helping a user work with Blueprint, a system for composing Airflow DAGs from YAML using reusable Python templates. Execute steps in order and prefer the simplest configuration that meets the user's needs.

...

GET IT FROM: <https://github.com/astromer/agents>

Guardrails with hooks



.claude/settings.json

```
{  
  "hooks": {  
    "PreToolUse": [{  
      "matcher": "Write|Edit",  
      "hooks": [{  
        "type": "command",  
        "command": ".claude/hooks/no-raw-dags.sh"  
      }]  
    }],  
    "PostToolUse": [{  
      "matcher": "Write|Edit",  
      "hooks": [{  
        "type": "command",  
        "command": ".claude/hooks/lint.sh"  
      }]  
    }]  
  }  
}
```



.claude/hooks/lint.sh

```
#!/usr/bin/env bash  
f=$(jq -r '.tool_input.file_path')  
[[ "$f" == *.dag.yaml ]] || exit 0  
  
uvx --from airflow-blueprint blueprint lint "$f" \  
|| { echo "blueprint lint failed" >&2; exit 2; }
```

Abstraction!

ASTRONOMER

Abstraction *enables* everyone



Marketing

Daily attribution reports



CRM

Personalized newsletter



Analytics

Third-party data imports

The person who needed a pipeline at the start of this talk **still cannot write Python**

“

EVERY ABSTRACTION IS A **DECISION** ABOUT **WHO GETS TO PARTICIPATE**.
THE POWER OF A SYSTEM IS NOT MEASURED BY THE COMPLEXITY IT CAN
HANDLE, BUT **BY THE NUMBER OF PEOPLE IT CAN INCLUDE**.

Thank you

For more Airflow madness:

[linkedin.com/in/vjanz](https://www.linkedin.com/in/vjanz)



ENJOYED THE TALK? TAG ME!



ASTRONOMER

The 2026 Apache Airflow® Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)

